

Лекция 10

Понеделник
18-21

в 210 ФХФ

- лекциите следващите три

Move семантики

$A \&\&$ - *rvalue* референция

$\&(A \&\& \text{ref})$ → Обектът ще умре малко след функцията

$A(A \&\&)$
 $A\& \text{OP} = (A \&\& \text{ref})$

колекция: // обектът трябва да живее независимо от останалите обекти
 $\text{add}(\text{const } A\&)$
 $\text{add}(A \&\&)$
 $\cdot \text{add}(\text{obj})$

$\{ A \text{ obj}; \quad A()$
 $\quad f(\text{std::move}(\text{obj}));$ // нищо $f(A \&\&)$ { } → референция
 $\quad g(\text{std::move}(\text{obj}));$ // М.С и $g(A \text{ obj})$ { }
 $\quad g(\text{obj});$ // С.С. $\sim A$ { $\sim A()$ move конструктор
 $\} \sim A$

std::move // преобразува от *lvalue* към *rvalue* (това е частен случай на *rvalue*)

$A(\text{const } A\&);$ - приема *lvalue* и *rvalue*
 $A(A \&\&);$ - приема *rvalue*

→ move конструктор
компилятор → A(A&&)

```
class A  
{ X obj;  
  Y obj2;  
}
```

// Ако в този клас няма
копир конструктор, компилаторът
ни прави move конструктор

A(const A&) = delete // Ако забраним КК, ще
⇒ A(A&&) = delete се забрани и МС

A(A&&) other:

```
obj(std::move(other.obj));  
obj2(std::move(other.obj2));  
}
```

} това го
прави
компиляторът

→ Ако има — move, ако няма — copy.

X(const X&) = default;

X(X&&) = delete;

// забраним се този ⇒ трябва да
напишем СС

OP = (A&& other)

{ if (this != &other)

{ free();

obj = std::move(other.obj);

obj2 = std::move(other.obj2);

;

return *this;

}

} това го
прави
компиляторът

Наследяване

def: Der е наследник Base,
ако разширява неговите данни и поведение.

Пример: class Base {
public:
int x;
f();
private:
int y;
g();
}

class Der: public Base
{ public
int z;
}

Der obj; // този обект има всичко, което има Base
obj.z++;
obj.x++;
obj.f();



Каква е разликата между композицията и наследяването?
// има разлика семантично

class DerTest {
Base b;
}

DerTest obj;
obj.b.z++;
obj.b.x++;
obj.b.f();
DerTest

Base	.z
------	----

композиция — has a relationship
наследяване — is a relationship

Пример:

```
class Dog {  
    char* name;  
    int age;  
}
```

```
class Person {  
    char* name;  
    int age;  
}
```

правим си клас за тези променливи

```
class Base {
```

```
    public:
```

```
        int x;
```

```
        f();
```

```
    private:
```

```
        int y;
```

```
}
```

```
class Der: public Base
```

```
{  
    g() {
```

```
        f(); x--;
```

```
        y++;
```

```
    }  
    int x;
```

— това не може,
защото y е private за
Base



публичния интерфейс на Base.

Der не се интересува как Base работи.

Base::x--; → достъп до x на Base

Видове наследяване — три вида

→ Private

→ Protected

→ Public

Пример: class X {

private:

int a;

protected: // наследниците могат да го достъпят,
вънният свят не

int b;

public:

int x;

}

class A: public X {

// публично наследяване

a - наследява се, но нямаме достъп в A

b - protected остава

x - public си остава

}

class B: protected X {

// protected наследяване

a - нямаме достъп

b - protected

x - ~~public~~ protected

}

class C: private X {

// private наследяване

a - нямаме достъп

b - private

x - private

}

X

{ A: private X

B: public A } // нямаме достъп

}

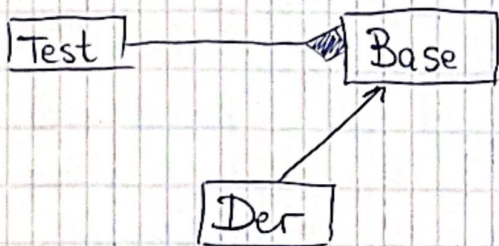


При класовете по default наследяването е private.

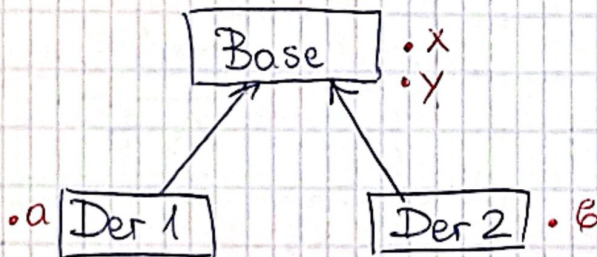
При структурите по default наследяването е public.

При композиция:

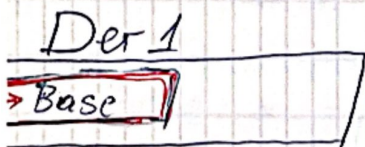
При наследяване:



При наследяване можем да "насохваме" указател/референция от базовия клас към наследника.



... // какви променливи има



Der 1 obj:

Base * ptr = &obj;

// обектът Base е в началото и можем

Base &ref = obj;

ptr не порозира за съществуването на Der 1

ptr -> x++;
ptr -> y++;

ptr -> a++;

не може // губи се конкретика


```

f(const Base* ptr)
  ptr → x;
  ptr → y;

```

```

Base 01;
Der1 02;
Der2 03;
f(&01);
f(&02);
f(&03);

```

} ще бъдат
валидни

```

f(Der1* ptr)

```

```

Der1 obj;
Base* ptr = &obj;
f(ptr);

```

x // няма да се компилира
опитва се да го
каже нагоре



```

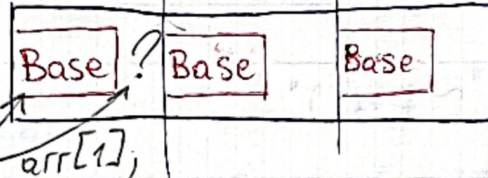
Der1 arr[3];

```

```

f(arr, 3) // компилира се

```



```

f(const Base* p arr, size)
  arr[1];

```

// защото Base/Base/Base не е така,
а между Base има padding

• Конструктори при наследяване

→ Конструкторът на наследника трябва да каже, кой конструктор на базовия клас да извика.
Ако не каже, се извиква default-ния.

```

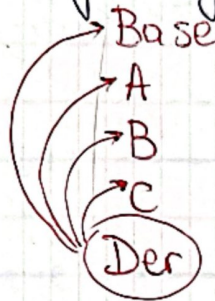
Der(int x, int y): Base(x) // да кажем, че x е от Base
{
  y... // правим нещо с y
}

```


class Der: Base

A
B
C

Der(...): obj 2, obj 1, Base....



извикването на
конструкторите зависи
от инициализацията
на списък

• Деструктори при наследяване

→ Деструкторът на наследникът извиква
деструкторът на базовия клас.

```
~Der() {  
    cout << "~Der()";  
}
```

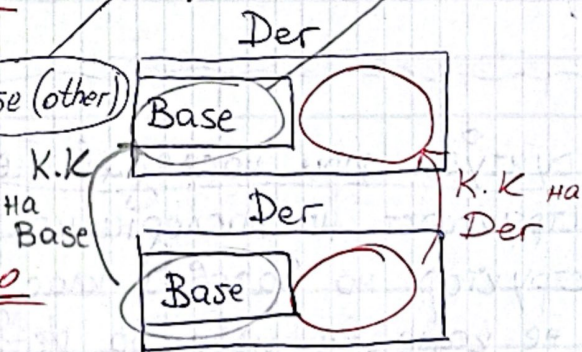
{ } ~C() ~B() ~A() ~Base
Base(const Base &)

• Копиране при наследяване

```
Der(const Der & other): Base(other)
```

{
 copyFrom(other);
}

призива се само за Der



• Оператор = при наследяване

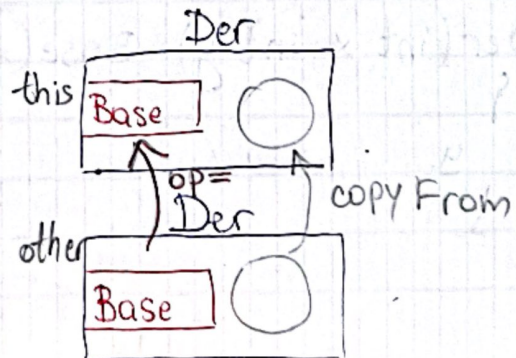
```
OP=(const Der & other)  
{ if (this != &other)
```

{ free();

copyFrom(other);

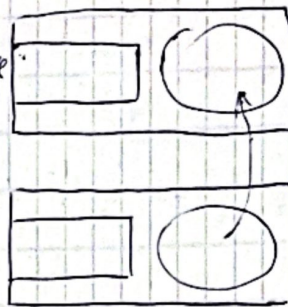
Base::operator=(other);

три
разични
на Der



Move при наследяване

```
Der (Der & other): Base(std::move(
    { moveFrom (std::move(other));
```



moveFrom (Der &&)

OP = (Der && other)

```
{ if (...)
  { free();
    moveFrom (std::move(other));
    Base:: OP = (std::move(other));
  }
  return *this;
}
```

отпегат
в началото

отпегат
в края

При теоретични задачи.

- може да има отпегатване в началото
- отпегатване в края

Base

A obj1;
B obj2;

```
{ Der d;
  }
```

// A(), B(), Base(),
X(), Y(), Der()

~Der() ←
~Y() ←
~X() ←
~Base() ←
~B() ←
~A() ←

Der

X obj1;
Y obj2;

// отпегатване в началото на OP = и на Der и на Base

Der d1; // създават се всички констр. като горните

Der d2; // —

d1 = d2; // OP = Der, OP = Base, OP = A, OP = B, OP = X, OP = Y

no default: Base OP =
free()
copy From (..)

- В края на Der и Base
A, B, Base, X, Y, Der

- В края на Der и началото на Base

- В края на Base и началото на Der

Пример за изпита:

Човек

- име (с произв. рѣжника)
- години

Студент

- име
- години
- орн

Преподавател

- име
- години
- предмети, които преподава

